

T iLi uK Thu t - H Th ng M R ng

Phi n b n PDF c t o t t i l i u Markdown hi n c trong workspace.

T iLi uK Thu t - H Th ng M R ng

T i l i u n y m t k i n t r c , c u h n h c t l i v quy t r n h c i t l i m i t r n g n theo n g m c t i u :
k i m t h replication MySQL , sau m r n g sang 2 API node v Nginx load balancing .

1. M cTi uThi tK

H th ng c x y d n g theo h ng n g i n n h ng r demo:

- € Client ch i m v i c v i Nginx .
- € Nginx ph n ph i request sang 2 API node .
- € API ghi d , li u v o MySQL Master .
- € API f c d , li u t MySQL Slave .
- € Slave nh , n replication t Master ch ... n g minh d , li u ghi v o Master c th quan s t Slave .

2. K i n Tr c H Th ng

H th ng d t n g Docker Compose v i 5 th nh ph n ch n h :

- € 1 Nginx l m entry point duy nh t .
- € 2 API node ch y c t n g m t m % ngu n Node.js .
- € 1 MySQL Master ghi d , li u .
- € 1 MySQL Slave nh , n replication v ph c v truy v n f c .

S minh h f a k i n t r c t n g th :

```
flowchart TB
    Client[Client / Postman / curl]
    Nginx[Nginx Load Balancer\nEntry point :8080]

    subgraph API[T ng API]
        A1[API Node 1\nNode_1 :3000]
        A2[API Node 2\nNode_2 :3000]
    end

    subgraph DB[T ng Database]
        M[MySQL Master\nWrite: :3306]
        S[MySQL Slave\nRead replica: :3306 internal]
    end

    Client --> Nginx
    Nginx --> A1
    Nginx --> A2

    A1 -->|POST /products| M
    A2 -->|POST /products| M

    A1 -->|GET /products| S
    A2 -->|GET /products| S

    M -->|Replication| S
```

Luồng xử lý chính

€ POST /products ghi vào Master.

€ GET /products fetch Slave.

€ Nginx tiếp nhận request từ 2 API node theo upstream round robin.

€ Mỗi API node trả về processed_by chỉ ra đang xử lý request nào đi qua node nào.

Số lượng yêu cầu được trả về trong bảng có thể khác nhau trong video vì thời gian thực: client, API và database.

3. Cấu trúc Hình Thức

3.1. Docker Compose

File: docker-compose.yml

```
services:
  mysql_master:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: products_db
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    ports:
      - "3306:3306"
    volumes:
      - ./db/master-init.sql:/docker-entrypoint-initdb.d/init.sql
      - mysql_master_data:/var/lib/mysql
      - ./db/master.conf:/etc/mysql/conf.d/master.conf
    command: >
      --server-id=1
      --log-bin=mysql-bin
      --binlog-format=ROW

  api_node_1:
    build:
      context: ./app
    environment:
      PORT: 3000
      SERVER_ID: Node_1
      MASTER_HOST: mysql_master
      MASTER_PORT: 3306
      SLAVE_HOST: mysql_slave
      SLAVE_PORT: 3306
      DB_USER: app_user
      DB_PASSWORD: app_password
      DB_NAME: products_db
    ports:
      - "3001:3000"

  api_node_2:
    build:
      context: ./app
    environment:
      PORT: 3000
      SERVER_ID: Node_2
      MASTER_HOST: mysql_master
      MASTER_PORT: 3306
      SLAVE_HOST: mysql_slave
      SLAVE_PORT: 3306
      DB_USER: app_user
      DB_PASSWORD: app_password
```

```

    DB_NAME: products_db
  ports:
    - "3002:3000"

  nginx:
    image: nginx:1.27-alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro

  mysql_slave:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: products_db
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_password
    ports:
      - "3307:3306"
    volumes:
      - ./db/slave-init.sql:/docker-entrypoint-initdb.d/init.sql
      - mysql_slave_data:/var/lib/mysql
      - ./db/slave.conf:/etc/mysql/conf.d/slave.conf
    command: >
      --server-id=2
      --relay-log=mysql-relay-bin
      --relay-log-index=mysql-relay-bin.index
      --read-only=ON

```

3.2. Nginx Load Balancer

File: nginx/nginx.conf

```

events {
    worker_connections 1024;
}

http {
    upstream api_backend {
        server api_node_1:3000 max_fails=2 fail_timeout=5s;
        server api_node_2:3000 max_fails=2 fail_timeout=5s;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://api_backend;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_connect_timeout 2s;
            proxy_read_timeout 30s;
            proxy_send_timeout 30s;
            proxy_next_upstream error timeout http_502 http_503 http_504;
        }
    }
}

```

3.3. MySQL Master

File: db/master.conf

```
[mysqld]
server-id=1
log-bin=mysql-bin
binlog-format=ROW
log-slave-updates=ON
```

File: db/master-init.sql

```
CREATE USER IF NOT EXISTS 'repl'@'%' IDENTIFIED WITH mysql_native_password BY 'repl_password';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
FLUSH PRIVILEGES;
```

```
CREATE TABLE IF NOT EXISTS products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

3.4. MySQL Slave

File: db/slave.conf

```
[mysqld]
server-id=2
relay-log=mysql-relay-bin
relay-log-index=mysql-relay-bin.index
read-only=ON
```

File: db/slave-init.sql

```
CREATE TABLE IF NOT EXISTS products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

3.5. API Read/Write Splitting

File: app/src/server.js

```
const masterPool = mysql.createPool({
  ...commonPoolOptions,
  host: process.env.MASTER_HOST || '127.0.0.1',
  port: Number(process.env.MASTER_PORT || 3306)
});

const slavePool = mysql.createPool({
  ...commonPoolOptions,
  host: process.env.SLAVE_HOST || '127.0.0.1',
  port: Number(process.env.SLAVE_PORT || 3307)
});

app.post('/products', async (req, res) => {
  const [result] = await masterPool.execute(
    'INSERT INTO products (name, price) VALUES (?, ?)',
    [name.trim(), numericPrice]
  );

  const [rows] = await masterPool.execute('SELECT * FROM products WHERE id = ?', [result.insertId]);
```

```

    res.status(201).json({
      message: 'Product created successfully',
      processed_by: serverId,
      data: rows[0]
    });
  });

app.get('/products', async (_req, res) => {
  const [rows] = await slavePool.query(
    'SELECT id, name, price, created_at, updated_at FROM products ORDER BY id ASC'
  );

  res.json({
    processed_by: serverId,
    source: 'slave',
    data: rows
  });
});

```

3.6. API Dependencies

File: app/package.json

```

{
  "dependencies": {
    "dotenv": "^16.6.1",
    "express": "^4.21.2",
    "mysql2": "^3.14.3"
  }
}

```

4. Hướng Dẫn Cài Đặt Vĩ Kim Th

Phần này cung cấp hướng dẫn cài đặt demo về các thành phần video học tập minh chứng.

Bộ 1: Chuẩn bị môi trường

Bên cạnh:

- € Docker Desktop
- € Docker Compose
- € PowerShell hoặc terminal tùy chọn

Lưu ý quan trọng:

- € Docker Desktop phải được cài đặt trước khi chạy `docker-compose up -d --build`.
- € Nếu gặp lỗi khi cài đặt trong `pipe:///./pipe/dockerDesktopLinuxEngine`, hãy mở Docker Desktop và khởi động lại engine chuyển sang trạng thái `running`.
- € Nếu Docker không hoạt động trên Windows containers, hãy chuyển sang Linux containers và tạo một container mới với hình ảnh Linux như `mysql:8.0` và `nginx:1.27-alpine`.

Bộ 2: Khởi tạo môi trường

Chạy lệnh cài đặt:

```
docker-compose up -d --build
```

- **ngành a c a b c n y:**

€ T o 1 MySQL Master.

€ T o 1 MySQL Slave.

€ T o 2 API node.

€ T o Nginx l m c s ng v o duy nh t.

B c 3: Ki m tra c c service l n ch a

`docker-compose ps`

K' v f ng th y c c service:

€ `mysql_master`

€ `mysql_slave`

€ `api_node_1`

€ `api_node_2`

€ `nginx`

N u m t service ch a ch y, ki m tra log b' ng:

`docker-compose logs mysql_master`

`docker-compose logs mysql_slave`

`docker-compose logs api_node_1`

`docker-compose logs api_node_2`

`docker-compose logs nginx`

B c 4: Ki m tra nhanh gateway qua Nginx

G f i health endpoint nhi c u l n x c nh n request i qua gateway:

`Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5`

`Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5`

`Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5`

K t qu mong i:

```
{
  "status": "ok",
  "processed_by": "Node_1"
}
```

L u Ź: `processed_by` c th l Node_1 ho c Node_2 t t y request Nginx chuy n t i node n o.

B c 5: C u h nh replication cho MySQL Slave

L y tr ng th i Master:

`docker-compose exec mysql_master mysql -uroot -proot123 -e "SHOW MASTER STATUS;"`

T k t qu , l y hai gi tr c h n h:

€ File

€ Position

V`d :mysql-bin.000005 v 157

D ñg replica IO thread tr c khi Ši c u h nh:

```
docker-compose exec mysql_slave mysql -uroot -proot123 -e "STOP REPLICATION IO_THREAD FOR CHANNEL '';"
```

Thi t l,pl i ngu n ñg b :

```
docker-compose exec mysql_slave mysql -uroot -proot123 -e "CHANGE MASTER TO MASTER_HOST='mysql_master', MASTER_USE"
```

Kh i ñg replica:

```
docker-compose exec mysql_slave mysql -uroot -proot123 -e "START REPLICATION;"
```

Ki m tra tr ñg th i replication:

```
docker-compose exec mysql_slave mysql -uroot -proot123 -e "SHOW REPLICATION STATUS\G"
```

K' v f ñg th y:

€ Replica_IO_Running: Yes

€ Replica_SQL_Running: Yes

€ Seconds_Behind_Source: 0

B c 6: Demo ghi d li u v o Master

G i request POST qua gateway:

```
$body = @{ name = 'Video Demo Item'; price = 123.45 } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8080/products -ContentType 'application/json' -Body $body | C
```

K t qu mong i:

```
{
  "message": "Product created successfully",
  "processed_by": "Node_2",
  "data": {
    "id": 6,
    "name": "Video Demo Item",
    "price": "123.45"
  }
}
```

processed_by c th l Node_1 ho c Node_2, t t y node Nginx ch f n request .

B c 7: X c minh d li u v o Master

Ch y truy v n tr c ti p tr n Master:

```
docker-compose exec mysql_master mysql -uroot -proot123 products_db -e "SELECT * FROM products WHERE name='Video D
```

K t qu mong il c b ñghi v a ch"n.

N u mu n ki m tra tŠng th d , li u trong b ng:

```
docker-compose exec mysql_master mysql -uroot -proot123 products_db -e "SELECT id, name, price, created_at, update
```

B c 8: G€i GET €c t• Slave

```
Invoke-RestMethod http://localhost:8080/products | ConvertTo-Json -Depth 5
```

Khi quay video, c±n nh n m nh 2 i m:

€ ph n h i c source: slave

€ b n ghi v a POST xu t hi n trong danh s ch GET

B c 9: Ki m tra h th ng ph, n ph i request gi a 2 API node

Gfi nhi<u l±n endpoint health:

```
Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5
```

Khi quay video, gi , m n h nh terminal th y processed_by Ši qua l i gi , a Node_1 v Node_2.

B c 10: Chaos test - d•ng m t node API

D ng th c ng m t node:

```
docker-compose stop api_node_1
```

Gfi l i request qua Nginx:

```
Invoke-RestMethod http://localhost:8080/health | ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:8080/products | ConvertTo-Json -Depth 5
```

K t qu mong i:

€ request v•n tr v< b nh th ng

€ processed_by s—l Node_2

€ h th ng kh ng b" d ng d† m t node API %t•t

Kh i ng l i node sau khi test xong:

```
docker-compose start api_node_1
```

B c 11: D€n dfp khi c,,n

N u mu n d ng to n b h th ng:

```
docker-compose down
```

N u mu n x a lu n d , li u volume ch y l i t ±u:

`docker-compose down -v`

5. Ghi Ch K Thu t

- € Nginx l entry point duy nh t cho client.
- € API s d ng hai connection pool ri ng cho Master v Slave.
- € Master x lŽ ghi, Slave x lŽ f c.
- € Slave c t read-only tr nh ghi nh m.
- € processed_by trong response đ t ng ch...ng minh request % i qua node n o.

6. K t Lu n

Thi t l, p n y p h t h p v i m c ti u n v :

- € Load balancer ho t ng l m i m v o duy nh t.
- € 2 API node ch y song song tr n c t ng m ngu n.
- € MySQL Master-Slave replication c c u h nh r r ng.
- € API % t ch lu ng f c v ghi d demo.
- € H th ng c th ki m th l i b'ng c ch d ng m t API node m v n duy tr ho t ng.